

GERENCIAMENTO DE PROCESSOS

Disciplina: Sistema operacionais

Professor: Clovis Ferraro

Grupo: 2

SUMARIO

1.	INTRODUÇÃO	1
1.	METADOLOGIA	2
3.	ANALISE DOS COMANDO E CONFIGURAÇÕES:	7
3.1.	<i>Linux</i>	7
3.2.	<i>Windows:</i>	17
3.3.	<i>Android:.....</i>	22
4.	ANALISE CRITICA:	26
4.1.	<i>Tabela comparativa:</i>	26
4.2.	<i>Discussão.....</i>	27
5.	CONCLUSÃO	28
6.	AUTOAVALIAÇÃO	28

1. INTRODUÇÃO

O objetivo deste relatório é implementar e analisar comandos de gerenciamento de processos nos sistemas Windows, Linux e Android para entender o ciclo de vida do processo, o controle de execução e também a estrutura de threads em cada ambiente. O gerenciamento é um componente importante para a estabilidade, porque ele garante que os processos não interfiram uns nos outros, ou seja, mantém o controle sobre as tarefas que estão em execução, garantindo que cada processo receba um tempo de CPU justo.

Para o desempenho do sistema, ele é muito importante porque ao terminar um processo, ocorre uma troca rápida, dando a ideia que mais de um processo estão sendo executados simultaneamente. E garante também a eficiência, ao usar o máximo da CPU, mas sem desperdícios. Quando o SO utiliza escalonamento para otimizar o uso da CPU, o escalonador equilibra o desempenho, gerencia o tempo de processamento dos processos e minimiza o tempo de espera de forma eficiente. Um processo nada mais é do que um programa em execução e thread é a menor unidade de processamento dentro de um processo. E a diferença deles é que o processo é a execução completa do programa, enquanto a thread é uma parte interna dele que executa uma tarefa específica. E o SO gerencia os seus estados por meio do Bloco de Controle de Processo (BCP), ou Process Control Block (PCB), que é uma estrutura que armazena os dados de um processo.

1. METADOLOGIA

Sistemas operacionais utilizados:

Durante os testes e experimentos de gerenciamento de processos, foram utilizados os seguintes sistemas operacionais:

- Windows 11 Pro — ambiente principal para testes com PowerShell e Gerenciador de Tarefas.
- Ubuntu 22.04 LTS — utilizado para comparação prática de comandos equivalentes em sistemas Linux.
- Android 13 — acessado via ADB Shell, para observação do gerenciamento de processos em um sistema móvel baseado em Linux.

Esses três ambientes permitiram compreender diferentes formas de controle, monitoramento e manipulação de processos.

Ferramentas Utilizadas :

No Windows:

- Get-Process — exibe todos os processos ativos, com detalhes de uso de CPU, memória e PID.
- Stop-Process — encerra processos a partir do nome ou ID.
- Gerenciador de Tarefas (Task Manager) — ferramenta gráfica para observação e gerenciamento direto dos processos.

No Linux/Android:

- ps, top e htop — listagem e monitoramento em tempo real de processos.
- kill — encerramento de processos.
- nice / renice — alteração da prioridade de execução.
- adb shell — acesso remoto ao terminal do dispositivo Android, permitindo o uso de comandos nativos do sistema. **Monitoramento e Identificação de Processos**

No Windows, utilizou-se o comando:

- Get-Process

```

Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.
Experimente a nova plataforma cruzada PowerShell https://aka.ms/powershell

PS C:\Users\shokt> get-process

Handles NPM(K) PM(K) VS(K) CPU(s) Id SI ProcessName
-----
231 8 2848 12128 0,47 6640 3 smss
148 8 4816 9628 0,13 6476 0 AggregatorHost
232 30 9236 20568 0,13 15606 3 ApplicationFrameHost
195 12 7680 13788 276,14 2376 0 Audiodg
342 12 12112 13712 1,37 1440 3 backlase
15579 526 573064 371916 10,513,86 7136 3 backlase
68 5 2408 5648 0,82 1832 3 cmd
143 7 1688 9440 0,80 336 3 CompAggSrv
196 12 9244 18116 0,45 784 3 conhost
274 14 4252 16984 0,22 8272 3 conhost
141 14 9608 16960 12,23 944 3 csrss
667 22 2844 6168 636 0 csrss
862 3 2844 7912 636 0 csrss
549 18 4620 22532 10,42 13592 3 ctfmon
315 14 5644 15376 0,59 2116 3 dlhst
215 18 4132 11460 5384 0 dlhst
1344 64 13160 25080 1880 0 explorer
3688 135 290356 253780 138,42 1652 3 explorer
172 13 2508 10480 1,73 364 3 famitr
100 8 1452 6492 1,73 10516 3 famitr
213 17 9788 27488 0,14 6480 3 flicdch
58 7 1588 3724 532 0 fntdrvhost
167 9 3636 8844 10836 0 GameInputRedistService
874 13 2416 45376 12232 0 gamingservices
269 14 11820 7680 8984 0 gamingservicesnet
216 17 26744 18232 0,44 9384 3 LightsOut
592 31 78832 130668 30,69 1748 3 Live!
610 30 61268 53784 0,59 14428 3 LockApp
1773 28 18072 26228 884 0 memcomp
0 0 1072 189360 2244 0 Memory Compression
149 9 2176 8848 0,86 12628 3 NvContainerCoreService
296 24 80952 121416 3,34 916 3 nvidiaeventview2
149 9 2176 8848 0,86 12628 3 nvidiaeventview2
525 24 78488 67232 1,77 13712 3 nvidiaeventview2
378 41 12332 97884 0,61 14224 3 nvidiaeventview2
194 11 9528 21188 0,11 15472 3 nvidiaeventview2
1125 47 48428 10188 3,16 15664 3 nvidiaeventview2
997 226 389880 311328 6564 0 nvidiaeventview2
286 18 48428 10188 3,16 15664 3 nvidiaeventview2
551 26 12992 32012 3480 0 nvidiaeventview2
342 17 8672 20176 2528 0 nvidiaeventview2
693 24 19680 41632 3,47 2528 3 nvidiaeventview2
688 47 5488 9912 40,97 1772 3 nvidiaeventview2
454 22 24488 32480 1772 0 nvidiaeventview2
617 28 28288 35768 18116 0 nvidiaeventview2
329 24 83312 51880 0,23 788 3 nvidiaeventview2
242 15 8676 10420 0,14 9480 3 nvidiaeventview2
1394 56 131608 118468 0,63 12188 3 nvidiaeventview2
381 21 12852 31384 1,64 13248 3 nvidiaeventview2
215 12 3808 12548 0,44 9788 3 nvidiaeventview2
1158 63 126992 125544 23,98 7928 3 nvidiaeventview2
451 26 37484 81992 1,14 18980 3 nvidiaeventview2
425 26 38424 81992 1,14 18980 3 nvidiaeventview2
832 64 366780 386816 113,42 6292 3 opera
645 31 70816 117888 2,69 9432 3 opera
57 15120 292116 38948 8736 3 opera
2180 70 157880 271152 217,46 10536 3 opera
231 14 13532 21808 217,46 10536 3 opera

```

Esse comando exibiu uma lista de processos com seus respectivos nomes, uso de CPU/memória e PID (Process ID) — número identificador único de cada processo. O PID também pôde ser visualizado na aba Detalhes do Gerenciador de Tarefas.

No Linux, o mesmo foi feito com:

- `ps aux`

e em tempo real com:

- `top`

onde cada linha representa um processo e a segunda coluna indica o PID.

Encerramento de processos:

No PowerShell, foram realizados testes com:

- `Stop-Process -Name <Nome>`
- `Stop-Process -Id <PID>`

Inicialmente, ao tentar encerrar o processo Taskmgr (Gerenciador de Tarefas), o sistema retornou:

Acesso negado.

```

PS C:\Users\shokt> stop-process -name taskmgr
stop-process : Não é possível interromper o processo "Taskmgr (18248)" devido ao seguinte erro: Acesso negado
No linha:1 caractere:1
+ stop-process -name taskmgr
+ ~~~~~
+ CategoryInfo          : CloseError: (System.Diagnostics.Process (Taskmgr):Process) [Stop-Process], ProcessComman
dException
+ FullyQualifiedErrorId : CouldNotStopProcess,Microsoft.PowerShell.Commands.StopProcessCommand

PS C:\Users\shokt>

```

Esse erro ocorreu porque o PowerShell estava sendo executado sem privilégios de administrador, e o processo era do sistema.

Após analisar a mensagem, foi compreendido que o PowerShell exige execução como administrador para manipular processos críticos.

Em seguida, o teste foi repetido com um processo comum e o comando funcionou corretamente:

- `Stop-Process -Name taskmgr`

```
PS C:\WINDOWS\system32> stop-process -name taskmgr
PS C:\WINDOWS\system32>
```

- `Stop-Process -Id <PID>`

```
S C:\WINDOWS\system32> stop-process -id 5272
S C:\WINDOWS\system32>
```

691	33	29940	56740	2,25	5272	3	Taskmgr
650	27	58484	53208	2,03	12156	3	TextInputHost
139	10	2168	10404	0,13	7720	3	User008EBroker
171	11	1692	7172	0,17	748	0	wininit
287	13	2712	11564	0,22	17548	3	winlogon
680	104	52524	109144	45,77	2400	3	wordpad
387	69	30136	70572	1,45	11820	3	wordpad
280	13	2096	10768	0,08	8196	0	WUDFHost
554	27	63772	14528	0,98	12936	3	XboxPcApp
381	19	7656	29092	0,69	8256	3	XboxPcAppFT

(ambos os processos funcionaram normalmente após a execução do powershell como administrador)

Isso confirmou o funcionamento do comando e a importância dos privilégios adequados.

Alteração de Prioridade :

No Linux, foi testada a execução de um processo com prioridade reduzida:

- ❑ `nice -n 10 gedit &`

E depois a modificação dessa prioridade com:

- ❑ `renice -n 5 -p <PID>`

Observou-se que, ao aumentar o valor de nice, o processo passa a ter menor prioridade na CPU, sendo preterido em situações de carga.

No Windows, esse ajuste pode ser feito graficamente pelo Gerenciador de Tarefas, clicando com o botão direito no processo → “Definir prioridade”.

Acesso e Manipulação em Android :

Com o dispositivo conectado via ADB, foram utilizados:

- `adb shell ps`

```
C:\Users\shokt\Desktop\Nova pasta (6)\platform-tools>adb devices
List of devices attached
RQCX406GGDL      device

C:\Users\shokt\Desktop\Nova pasta (6)\platform-tools>adb shell
a35x:/ $ ps
USER          PID  PPID      VSZ      RSS WCHAN          ADDR S NAME
shell         4809  4672    10773664  3160 __do_sys_+      0 S sh
shell         4879  4809    10840260  3768 0                0 R ps
a35x:/ $
```

- adb shell top

```
Tasks: 741 total, 3 running, 738 sleeping, 0 stopped, 0 zombie
Mem: 7615492K total, 7347200K used, 268292K free, 1548K buffers
Swap: 4194300K total, 2527292K used, 1667008K free, 2704808K cached
800%cpu 34%user 11%nice 32%sys 714%idle 2%iow 6%irq 1%siirq 0%host
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	ARGS
1144	system	18	-2	27G	390M	160M	R	16.3	5.2	231:16.94	system_server
23871	u0_a107	20	0	17G	119M	59M	S	14.6	1.5	0:03.99	com.samsung.android.scs
995	u0_a50	20	0	17G	136M	93M	S	13.3	1.8	0:06.60	com.android.systemui.edgellighting
797	system	-3	-8	11G	30M	17M	R	11.0	0.4	164:11.26	surfaceflinger
1557	u0_a50	20	0	19G	353M	145M	S	8.3	4.7	91:17.22	com.android.systemui
23011	u0_a56	20	0	17G	92M	53M	S	3.3	1.2	0:05.80	android.process.acore
721	system	-3	-8	11G	6.2M	5.1M	S	3.0	0.0	47:36.75	android.hardware.graphics.composer@2.4-service
4991	shell	20	0	10G	5.1M	3.4M	R	2.0	0.0	0:00.11	top
4887	u0_a116	20	0	18G	148M	85M	S	1.3	1.9	0:00.73	com.samsung.android.messaging
1600	root	20	0	0	0	0	I	1.3	0.0	0:02.23	[kworker/u16:3-blk_sec_common]
32066	root	0	-20	0	0	0	I	1.3	0.0	0:04.46	[kworker/u17:0-kbase_pm_poweroff_wait]
244	root	RT	0	0	0	0	S	1.3	0.0	21:06.74	[crtcd_kthread]
27702	system	20	0	16G	79M	43M	S	1.0	1.0	0:28.56	com.samsung.sait.sohservice
1006	system	20	0	11G	19M	7.6M	S	1.0	0.2	4:32.98	vendor.samsung.hardware.biometrics.fingerprint-service
462	logd	30	10	10G	11M	3.2M	S	1.0	0.1	16:22.11	logd
32422	root	0	-20	0	0	0	I	0.6	0.0	0:04.68	[kworker/u17:4-kbase_pm_poweroff_wait]
1933	system	20	0	16G	60M	33M	S	0.6	0.8	5:43.06	com.samsung.sec.android.teegris.tui_service
1541	radio	20	0	18G	116M	60M	S	0.6	1.5	21:01.52	com.android.phone
700	statsd	20	0	10G	6.5M	3.3M	S	0.6	0.0	6:54.20	statsd
170	root	RT	0	0	0	0	S	0.6	0.0	13:46.37	[fast_switch_post_worker_dm_cpuc10]
3850	u0_a120	20	0	17G	260M	235M	S	0.3	3.4	0:06.57	com.android.settings.intelligence
1681	root	20	0	0	0	0	I	0.3	0.0	0:00.88	[kworker/0:0-events]
1159	root	20	0	0	0	0	I	0.3	0.0	0:00.30	[kworker/3:4-pm]
31164	root	20	0	0	0	0	I	0.3	0.0	0:00.19	[kworker/5:0-events]
23335	u0_a108	20	0	18G	91M	63M	S	0.3	1.2	0:04.07	com.osp.app.signin

-

para visualizar a lista de processos ativos, verificando consumo de recursos e PIDs.

Também foi possível encerrar processos específicos usando:

- adb shell kill <PID>

Validação :

A validação foi feita com base em três testes práticos:

Encerramento de processos comuns e do sistema:

- Processos simples (como notepad) puderam ser encerrados normalmente.
- Processos do sistema (como taskmgr) retornaram erro de acesso negado, confirmando a necessidade de permissões elevadas.

Monitoramento em tempo real:

- O uso dos comandos top (Linux/Android) e Gerenciador de Tarefas (Windows) mostrou a variação de CPU e memória em tempo real.

Alteração de prioridade:

- A alteração via nice/renice (Linux) e via interface (Windows) teve efeito perceptível: o processo com menor prioridade teve desempenho reduzido sob carga do sistema.

Conclusão da Metodologia:

O experimento demonstrou de forma prática como cada sistema operacional gerencia seus processos, confirmando que:

- Cada processo possui um identificador único (PID), essencial para controle direto.
- O encerramento e a modificação de prioridade exigem diferentes níveis de permissão, dependendo do tipo de processo.
- O PowerShell e o terminal Linux oferecem comandos poderosos para administração e automação do controle de processos.
- Testes práticos com erros (como “Acesso negado”) foram úteis para compreender as limitações de segurança do sistema.

Em síntese, o gerenciamento de processos é um aspecto central do funcionamento dos sistemas operacionais, garantindo estabilidade, desempenho e segurança no uso dos recursos computacionais

3. ANALISE DOS COMANDO E CONFIGURAÇÕES:

3.1. Linux:

-PS

-Comando utilizado para exibir processos que estão em execução na máquina, além de mostrar ao usuário as informações dos mesmos.

```
File Edit View Terminal Tabs Help
labex:project/ $ ps
  PID TTY          TIME CMD
  328 pts/1        00:00:00 zsh
  351 pts/1        00:00:00 ps
```

🔍 PID é visto como o número de identificação (ou ID) de um processo em execução.

🔍 Utilizamos PPID para identificar o processo-pai.

Opções de Comandos (PS):



-a: Deixa em exibição os processos de todos os usuários.

```
labex:project/ $ ps -a
  PID TTY          TIME CMD
  327 pts/0        00:00:00 script
  665 pts/1        00:00:00 ps
```



-e: Mostra todos os processos.

```
labex:project/ $ ps -e
```

PID	TTY	TIME	CMD
1	?	00:00:00	init.sh
21	?	00:00:00	supervisord
22	?	00:00:00	sshd
23	?	00:00:00	su
24	?	00:00:00	vncserver
36	?	00:00:04	Xvnc
46	?	00:00:00	sh
47	?	00:00:00	sh
48	?	00:00:00	xfce4-session
57	?	00:00:00	dbus-launch
58	?	00:00:00	dbus-daemon
60	?	00:00:00	at-spi-bus-laun
65	?	00:00:00	dbus-daemon
69	?	00:00:00	xfconfd
75	?	00:00:00	at-spi2-registr
80	?	00:00:00	ssh-agent



-f: É mostrado para o usuário a árvore de execução dos comandos, basicamente a estrutura dos comandos.

```
labex:project/ $ ps -f
```

UID	PID	PPID	C	STIME	TTY	TIME	CMD
labex	328	327	0	10:12	pts/1	00:00:00	zsh -i
labex	813	328	0	10:46	pts/1	00:00:00	ps -f



-g(grupo): Será exibido os processos de acordo com determinado nome de grupo.

```
labex:project/ $ ps -g
```

PID	TTY	STAT	TIME	COMMAND
265	pts/0	Ss	0:00	zsh
327	pts/0	S+	0:00	script -q -f /tmp/session/1760580723.log
328	pts/1	Ss	0:00	zsh -i
852	pts/1	R+	0:00	ps -g



-o(formato): mostra apenas o formato definido pelo usuário.

Para cada informação ser exibida, deve-se usar palavras chaves como **pid**, **ppid**, **ni**, **pri** (prioridade) e **comm** (comando).

```

labex:project/ $ ps -o pid
PID
328
914
labex:project/ $ ps -o ppid
PPID
327
328
labex:project/ $ ps -o ni
NI
0
0
labex:project/ $ ps -o pri
PRI
19
19
labex:project/ $ ps -o comm
COMMAND
zsh

```

-x: Comando com a principal função de exibir todos os processos que ainda não foram iniciados no console.

```

labex:project/ $ ps -x
  PID TTY          STAT       TIME COMMAND
   24 ?            Ss          0:00 /usr/bin/perl /usr/bin/vncserver -fg -disableBasicAuth -publicIp 0.0.0.0
   36 ?            Sl          0:18 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.0.0.0 -Log *:stdout:100 -AcceptPoint
   46 ?            S           0:00 sh -c { echo 'Running /home/labex/.vnc/xstartup'; /home/labex/.vnc/xstartup; } >> '
   47 ?            S           0:00 sh -c { echo 'Running /home/labex/.vnc/xstartup'; /home/labex/.vnc/xstartup; } >> '
   48 ?            Sl          0:00 xfce4-session
   57 ?            S           0:00 /usr/bin/dbus-launch --sh-syntax --exit-with-session xfce4-session
   58 ?            Ss          0:00 /usr/bin/dbus-daemon --syslog --fork --print-pid 5 --print-address 7 --session
   60 ?            Sl          0:00 /usr/libexec/at-spi-bus-launcher
   65 ?            S           0:00 /usr/bin/dbus-daemon --config-file=/usr/share/default/at-spi2/accessibility.conf -
   69 ?            Sl          0:00 /usr/lib/x86_64-linux-gnu/xfce4/xfconf/xfconfd
   75 ?            Sl          0:00 /usr/libexec/at-spi2-registryd --use-gnome-session
   80 ?            Ss          0:00 /usr/bin/ssh-agent -s

```



-u: É fornecido o nome do usuário e a hora em que o processo teve início.

```
labex:project/ $ ps -u
```

USER	PID	%CPU	%MEM	VSZ	RSS	TTY	STAT	START	TIME	COMMAND
labex	265	0.0	0.0	14496	6692	pts/0	Ss	10:12	0:00	zsh
labex	327	0.0	0.0	9620	1120	pts/0	R+	10:12	0:00	script -q -f /tmp/session/1760580723.log
labex	328	0.0	0.0	14480	6616	pts/1	Ss	10:12	0:00	zsh -i

Observações:

- Podemos notar que o próprio comando ps cria um processo em si que já faz parte do resultado final do comando
- As prioridades exibidas pelo comando ps variam de -40 a 99, muito diferente do que é mostrado em outro comando de monitoramento de processos.
- Para conseguirmos ter visualização dos processos que estão em execução no sistema, o dire tório */proc* deve ser executado.

-top/htop:

```
Mem: 5696K used, 181548K free, 8K shrd, 0K buff, 872K cached
CPU:  0% usr  0% sys  0% nic 95% idle  0% io  0% irq  3% irq
Load average: 0.01 0.01 0.00 1/32 94
```

PID	PPID	USER	STAT	VSZ	%VSZ	CPU	%CPU	COMMAND
94	63	root	R	1516	1%	0	1%	top
63	1	root	S	1552	1%	0	0%	sh -l
1	0	root	S	1512	1%	0	0%	{init} /bin/sh /sbin/init
58	1	root	S	1260	1%	0	0%	dhcpcd -q
56	1	root	S	744	0%	0	0%	settime -d /
7	2	root	SW	0	0%	0	0%	[ksoftirqd/0]
6	2	root	SW<	0	0%	0	0%	[mm_percpu_wq]
8	2	root	SW	0	0%	0	0%	[kdevtmpfs]
4	2	root	SW<	0	0%	0	0%	[kworker/0:0H]
10	2	root	SW<	0	0%	0	0%	[writeback]
11	2	root	SW	0	0%	0	0%	[kcompactd0]
12	2	root	SW<	0	0%	0	0%	[crypto]
13	2	root	SW<	0	0%	0	0%	[bioset]
14	2	root	SW<	0	0%	0	0%	[kblockd]
2	0	root	SW	0	0%	0	0%	[kthreadd]
16	2	root	SW	0	0%	0	0%	[kswapd0]
17	2	root	SW<	0	0%	0	0%	[bioset]
34	2	root	SW	0	0%	0	0%	[khvcd]
35	2	root	SW<	0	0%	0	0%	[bioset]
36	2	root	SW<	0	0%	0	0%	[bioset]
37	2	root	SW<	0	0%	0	0%	[bioset]
38	2	root	SW<	0	0%	0	0%	[bioset]
39	2	root	SW<	0	0%	0	0%	[bioset]
40	2	root	SW<	0	0%	0	0%	[bioset]
3	2	root	SW	0	0%	0	0%	[kworker/0:0]
5	2	root	SW	0	0%	0	0%	[kworker/u2:0]



top: Esse comando exibe informações sobre os processos que estão em execução através de uma interface.

-Opções de Comando (top):



-d (número): Atualiza a tela após o número de segundos fornecido pelo usuário.

```
Mem: 5772K used, 181472K free, 8K shrd, 0K buff, 904K cached
CPU:  0% usr  0% sys  0% nic 95% idle  0% io  0% irq  3% sirq
Load average: 0.05 0.01 0.00 1/31 98
```

PID	PPID	USER	STAT	VSZ	%VSZ	CPU	%CPU	COMMAND
98	63	root	R	1516	1%	0	1%	top -d 5
63	1	root	S	1552	1%	0	0%	sh -l
1	0	root	S	1512	1%	0	0%	{init} /bin/sh /sbin/init
58	1	root	S	1260	1%	0	0%	dhcpcd -q
56	1	root	S	744	0%	0	0%	settime -d /
7	2	root	SW	0	0%	0	0%	[ksoftirqd/0]
15	2	root	SW	0	0%	0	0%	[kworker/0:1]
8	2	root	SW	0	0%	0	0%	[kdevtmpfs]
9	2	root	SW	0	0%	0	0%	[oom_reaper]
5	2	root	SW	0	0%	0	0%	[kworker/u2:0]
12	2	root	SW<	0	0%	0	0%	[crypto]
11	2	root	SW	0	0%	0	0%	[kcompactd0]
14	2	root	SW<	0	0%	0	0%	[kblockd]
13	2	root	SW<	0	0%	0	0%	[bioset]

(O número padrão

de segundos é 5, mas vai da vontade do usuário definir um padrão para que a tela se atualize).



-p (pid): Mostra apenas o processo com o PID especificado pelo usuário.

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
36	labex	20	0	743784	100508	58128	S	13.0	0.6	0:05.38	Xvnc



-h : Exibe informações sobre o comando

top ,basicamente

sendo um guia para o usuário.

```
labex:project/ $ top -h
procpns-ng 3.3.17
Usage:
top -hv | -bcEeHi0Ss1 -d secs -n max -u|U user -p pid(s) -o field -w [cols]
```

htop: Basicamente, uma versão do **top** com uma interface com mais cores e mais opções para o usuário, já mostrando em baixo o que cada tecla irá realizar no comando.

```

0[|||||] 13.2% Tasks: 33, 38 thr; 4 running
1[|||||] 11.8% Load average: 0.23 0.25 0.32
2[|||||] 13.2% Uptime: 364 days(!), 04:37:41
3[|||||] 10.7%
Mem[|||||] 2.02G/15.4G
Swp[|||||] 0K/0K

PID USER PRI NI VIRT RES SHR S CPU% MEM% TIME+ Command
36 labex 20 0 867M 104M 58296 S 31.7 0.7 0:27.84 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
209 labex 20 0 867M 104M 58296 R 9.9 0.7 0:07.22 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
211 labex 20 0 867M 104M 58296 R 9.9 0.7 0:07.26 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
210 labex 20 0 867M 104M 58296 R 9.2 0.7 0:07.23 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
634 labex 20 0 867M 104M 58296 S 0.7 0.7 0:00.60 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
1 root 20 0 11204 3732 3456 S 0.0 0.0 0:00.01 /bin/bash /etc/shiyanlou/sbin/init.sh
21 root 20 0 40824 28092 10648 S 0.0 0.2 0:00.62 /usr/bin/python3 /usr/bin/supervisord -n
22 root 20 0 15424 9436 7800 S 0.0 0.1 0:00.02 sshd: /usr/sbin/sshd -D [listener] 0 of 10-100
23 root 20 0 14040 4384 3840 S 0.0 0.0 0:00.00 su labex -c vncserver -fg -disableBasicAuth -pu
24 labex 20 0 40312 30568 6500 S 0.0 0.2 0:00.51 /usr/bin/perl /usr/bin/vncserver -fg -disableBa
39 labex 20 0 867M 104M 58296 S 0.0 0.7 0:00.00 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
40 labex 20 0 867M 104M 58296 S 0.0 0.7 0:00.18 /usr/bin/Xvnc :1 -disableBasicAuth -publicIp 0.
F1Help F2Setup F3Search F4Filter F5Free F6SortBy F7Nice F8Vice F9Kill F10Quit

```

kill/killall:

kill: Comando utilizado para enviar um sinal para determinado processo. Ele é usado geralmente para finalizar a execução de processos identificado pelo seu PID.

-Opções de Comando (kill):



-l: Dá ao usuário uma lista com o nome dos sinais que podem ser utilizados.

```

localhost:~# kill -l
1) HUP
2) INT
3) QUIT
4) ILL
5) TRAP
6) ABRT
7) BUS
8) FPE
9) KILL
10) USR1
11) SEGV
12) USR2
13) PIPE
14) ALRM
15) TERM
16) STKFLT
17) CHLD
18) CONT
19) STOP
20) TSTP

```



-l(número): fornece o nome do sinal através do número fornecido.

-s (sinal): Especifica qual sinal deve ser enviado, onde o sinal pode ser um número ou um nome.

```
localhost:~# kill -1 31
SYS
localhost:~# kill 1- 8
sh: invalid number '1-'
localhost:~# kill -1 31
SYS
localhost:~# kill -1 8
FPE
localhost:~# kill -1 17
CHLD
localhost:~# kill -1 64
64
```



```
~ $ ps
  PID TTY          TIME CMD
 13956 pts/0    00:00:00 /data/data/com.termux/files/usr/bin/bash
 28874 pts/0    00:00:00 ps
~ $ kill -9 28874
bash: kill: (28874) - No such process
```

ExemplodeSinais



SIGHUP(1): reinicializa um processo (o processo irá levar novamente seus arquivos de configuração).

SIGTSTP(20): Um dos mais usados, pois irá suspender a execução de um processo.

killall: Este comando envia um determinado sinal a um conjunto de processos que usam o mesmo nome.

-Opções de Comando (killall):



-e (nome): Procura pelo nome de um processo que seja igual (exato) ao nome fornecido. Se o nome possuir mais de 15 caracteres, apenas os primeiros 15 serão comparados.

```
labex:project/ $ killall -e unacid
unacid: no process found
```

(Como o nome do processo não existe, ele não conseguiu encontrar nada para comparar).



-i: Pede confirmação antes de matar um processo.



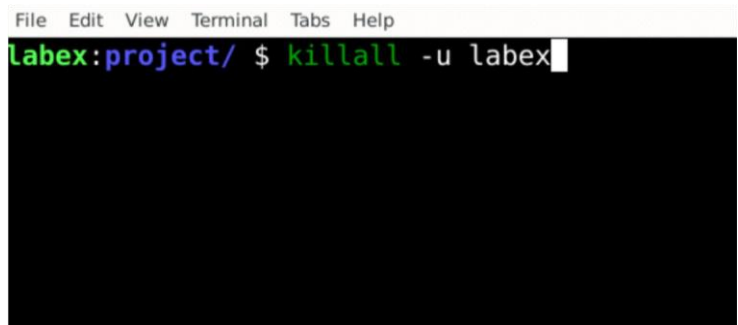
-l: Lista o conjunto de sinais que podem ser enviados junto ao comando **killall**.

```
labex:project/ $ killall -l
HUP INT QUIT ILL TRAP ABRT BUS FPE KILL USR1 SEGV USR2 PIPE ALRM TERM STKFLT
CHLD CONT STOP TSTP TTIN TTOU URG XCPU XFSZ VTALRM PROF WINCH POLL PWR SYS
```



-u

(usuário): Mata apenas os processos que estão vinculados a um usuário em específico.



```
File Edit View Terminal Tabs Help
labex:project/ $ killall -u labex
```

(Terminal Linux antes do código killall-u ser executado.)



(Logo após o comando ser executado, pois todos os processos estavam ligados a um único usuário.)



-v: Informa ao usuário se o sinal foi enviado corretamente e com sucesso.

-nice/renice:

nice: Define o valor de ajuste de prioridade na execução de um comando. A prioridade normal da execução de um processo padrão pode variar de 0 (número de maior prioridade) a 39 (número onde a prioridade é baixa). No caso, a prioridade é calculado como: **PR=15 + NI**. Onde **NI (nice)** é o valor usado para alterar a prioridade do processo.

```

Tasks: 2 total, 1 running, 1 sleeping, 0 stopped, 0 zombie
Mem: 5557580K total, 5433324K used, 124256K free, 852K buffers
Swap: 6291452K total, 3573088K used, 2718364K free, 1055304K cached
800%cpu 0%user 0%nice 0%sys 800%idle 0%iow 0%irq 0%sirq 0%host
  PID USER      PR  NI  VIRT  RES  SHR S[%CPU] %MEM    TIME+  ARGS
  23023 u0_a391    16   -4  10G  3.5M  2.8M R  0.3   0.0   0:00.06 top
  16645 u0_a391    10  -10  10G  2.4M  2.1M S  0.0   0.0   0:00.16 bash /data/data/c+

~ $ nice -5 top █

```

```

Tasks: 2 total, 1 running, 1 sleeping, 0 stopped, 0 zombie
Mem: 5557580K total, 5220496K used, 337084K free, 832K buffers
Swap: 6291452K total, 3778752K used, 2512700K free, 865956K cached
800%cpu 0%user 0%nice 0%sys 800%idle 0%iow 0%irq 0%sirq 0%host
  PID USER      PR  NI  VIRT  RES  SHR S[%CPU] %MEM    TIME+  ARGS
  23459 u0_a391    15  -5  10G  3.4M  2.7M R  0.0   0.0   0:00.03 top
  16645 u0_a391    10  -10  10G  2.3M  2.0M S  0.0   0.0   0:00.17 bash /data/data/c+

```

Note que após o uso do comando *nice*, a prioridade do top foi ajustada para o valor sugerido.

-Opções de Comando (nice):



-n (valor): Adiciona o valor especificado pelo usuário á prioridade de execução de um processo

```
~ $ nice -n 12 htop
```

```
26639 u0_a391 22 2 10.3G 5896 4376 R N/A 0.1 0:00.07 htop /data/data/
```

Pode se notar que a prioridade do comando htop foi alterada e somada conforme o número fornecido.



-help: Ajuda o usuário com uma lista de opções disponíveis com o comando.

```
labex:project/ $ nice --help
Usage: nice [OPTION] [COMMAND [ARG]...]
Run COMMAND with an adjusted niceness, which affects process scheduling.
With no COMMAND, print the current niceness.  Niceness values range from
-20 (most favorable to the process) to 19 (least favorable to the process).

Mandatory arguments to long options are mandatory for short options too.
  -n, --adjustment=N  add integer N to the niceness (default 10)
  --help              display this help and exit
  --version           output version information and exit

NOTE: your shell may have its own version of nice, which usually supersedes
the version described here.  Please refer to your shell's documentation
for details about the options it supports.

GNU coreutils online help: <https://www.gnu.org/software/coreutils/>
Full documentation <https://www.gnu.org/software/coreutils/nice>
or available locally via: info '(coreutils) nice invocation'
```



-- version : É mostrado a versão e informações sobre o comando.

```
labex:project/ $ nice --version
nice (GNU coreutils) 8.32
Copyright (C) 2020 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <https://gnu.org/licenses/gpl.html>.
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
```

renice: Este comando altera o valor *nice* usado para calcular a prioridade de um processo padrão

3.2. Windows:

O gerenciador de processos pode ser executando por (ctrl + shift + esc). Após executarmos o programa podemos observar que se trata de uma interface gráfica, com sua própria função de controlar, encerrar e monitorar processos. Assim desta maneira garantindo que os programas em execução funcionem de maneira correta sem atrapalhar os outros. Em suma nas versões mais recentes do Windows (10, 11), temos umas funções avançadas para gerenciar processos como:

- PI (Processo identifier): A identificação exclusiva do Kernal (em números) para rastrear o processo (chave primaria do EPROCESS)
- Definir processo em prioridade: Permite ao usuário alterar o nível de prioridade, assim informando ao escalonador o tempo total de CPU para respectivo processo.
- Definir afinidade: se permite selecionar em qual núcleo físico da CPU o respectivo processo pode ser executado. Isso e usado para fins de otimização ou até mesmo diagnostico.

Além dessas funções que diferenciam das versões anteriores do Sistema Operacional (SO) do Windows. Conforme temos como suas principais funções em demais versões anteriores: visão geral do sistema, Gerenciamento de processos, encerramento de tarefas, Monitoramento de desempenho, Inicialização do programa. Em conta disso foram citados algumas dessas funções mais em realidade as funções são divididas em quatro partes: Gerenciamento do Ciclo de Vida do Processo, escalonamento da CPU (*Scheduling*), Alocação de Recursos, interação e Coordenação. Como mostrado no mapa mental em seguida:

**GERENCIAMENTO
DE CICLO DE VIDA DE
PROCESSO:**

- Citação de processos
- Encerramento de processos
- Controle de estados

**ESCALONAMENTO
DA CPU:**

- Tomada de decisão
- Distribuição de tempo
- Priorização

**FUNÇÕES ESSENCIAIS
DE UM GERENCIADOR DE PROCESSOS**

**ALOCAÇÃO
E PROTEÇÃO DE
RECURSOS:**

- Alocação de memória
- Proteção de memória
- Gerenciamento E/S

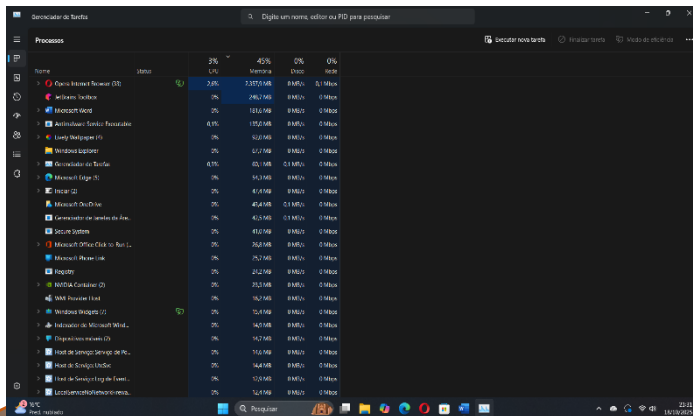
INTERAÇÃO E CORDENAÇÃO:

- Comunicação Inter processos (IPC)
- Sincronização
- Detecção e prevenção de Deadlock

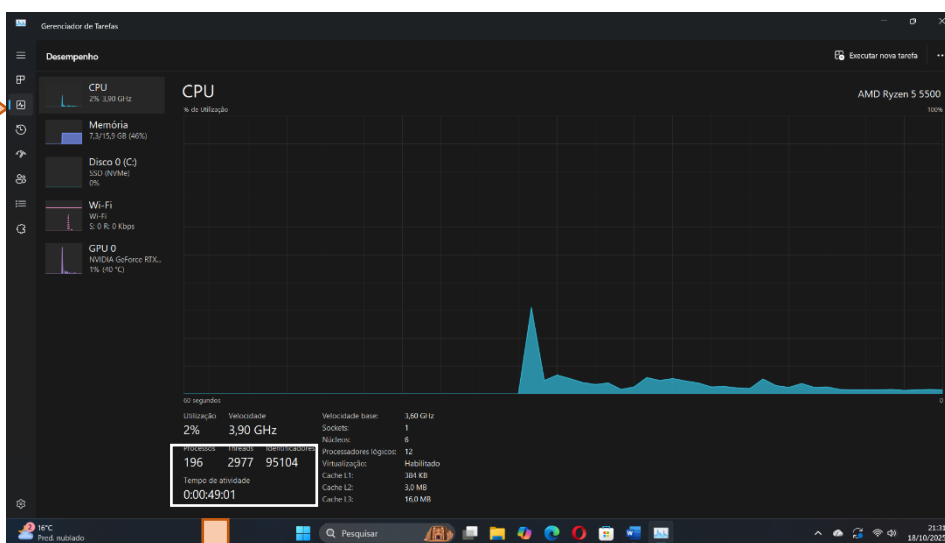
Ainda Pode-se gerenciar processos pelo power shell e o prompt de comando como os seguintes comandos:

- Get process
- Get-Process -Name chrome(Pode se colocar qualquer nome do programa)
- Stop-process -Name chrome
- Stop-process ID 1234

Por cima disso veremos um pouco da interface gráfica do gerenciar de processos para visualizar *Processos, desempenho e threads*.



Quando se realiza o processo que envolve o atalho CTRL+SHIFT+ESC, o gerenciador de tarefas é executado. Diretamente ele abre no painel de *processos* que por vez se visualiza todos os programas em execução.



Depois que damos um click em “desempenho” se é visualizada gráficos mais técnico para mostrar a situação de cada componente da máquina.

Utilização	Velocidade	
2%	3,90 GHz	
Processos	Threads	Identificadores
196	2977	
Tempo de atividade		
0:00:49:01		

Uma "linha de execução" dentro de um processo. Um processo pode ter várias threads trabalhando ao mesmo tempo. Assim mostrando quantos threads estão sendo usados.

São a quantidade de processos em execução na maquina

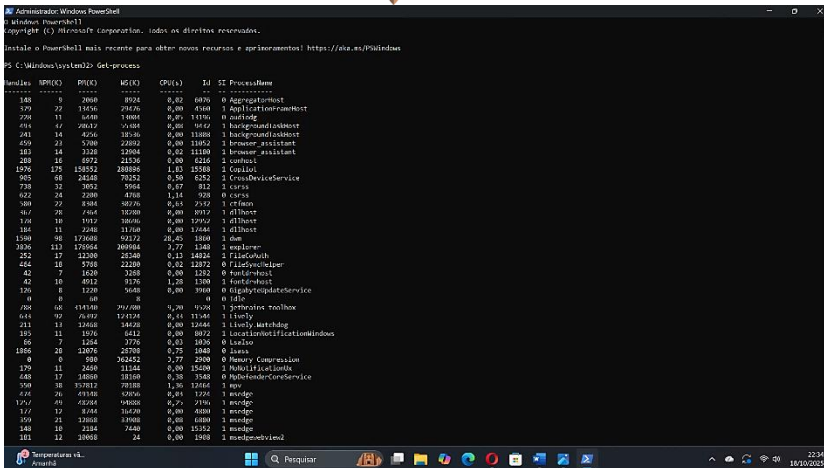
Por daqui em diante será mostrado os comandos e a funções respectivos de cada um (será usado o power shell).

```
Administrador: Windows PowerShell
O Windows PowerShell
Copyright (C) Microsoft Corporation. Todos os direitos reservados.

Instale o PowerShell mais recente para obter novos recursos e aprimoramentos! https://aka.ms/PSWindows

PS C:\Windows\system32> Get-process
```

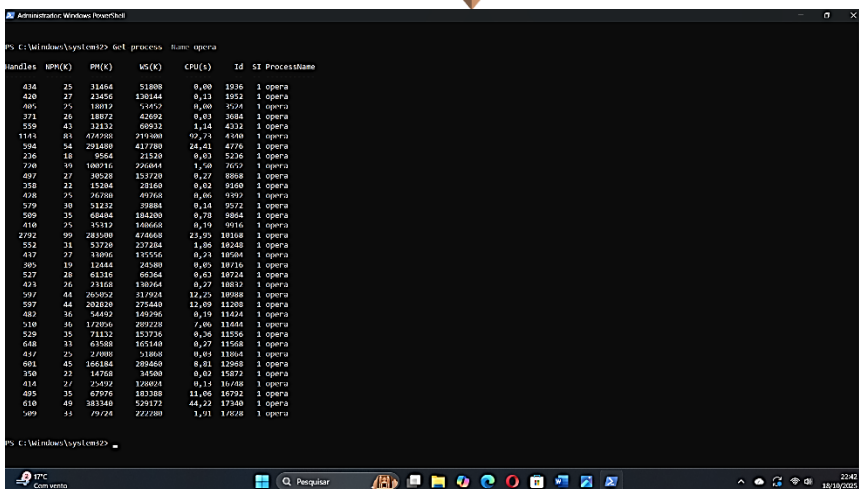
Usamos o comando Get-process para visualizar todos os processos executados em execução na CPU.



Handles	PM(K)	PM(K)	MS(K)	PM(K)	ID	SI	ProcessName
158	0	2608	8924	0,02	6806	0	System
179	22	14576	25436	0,06	4568	1	smss
276	11	6408	18084	0,01	11196	0	csrss
416	17	29832	52464	0,01	9472	1	backgroundcheck
241	14	4256	18236	0,00	11888	1	backgroundcheck
459	23	5700	22081	0,00	11892	1	browser assistant
183	14	3328	22904	0,02	11189	1	browser assistant
250	15	6072	21256	0,00	1216	1	cmd
1976	175	158552	288888	1,03	12560	1	Conhost
905	68	24832	76424	0,00	6282	1	CredentialService
770	32	3892	5964	0,07	812	1	csrss
622	24	2208	4768	1,14	928	0	csrss
380	22	8384	58276	0,51	2752	1	csrss
347	26	7464	18200	0,00	10972	1	dlhnt
116	18	1912	38608	0,00	12992	1	dlhnt
184	11	2248	31768	0,00	11644	1	dlhnt
1500	86	174408	92212	25,45	1808	1	explorer
3836	113	170664	269984	2,77	1340	1	explorer
252	17	12320	25240	0,12	14024	1	FileCch
404	18	5768	22180	0,02	12872	0	FileExchelper
42	7	1632	3280	0,00	12512	0	fontdrvhost
42	10	4912	9176	1,28	1308	1	fontdrvhost
126	8	1220	568	0,00	768	0	igmpv4update
0	0	0	0	0	0	0	lsass
788	68	41456	79720	4,70	9704	1	netproctoolbox
144	19	7492	19184	0,54	11144	1	lsass
211	11	1248	34420	0,00	1044	1	lsass
185	11	1076	4412	0,00	8072	1	lsass
66	7	1264	3776	0,02	1836	0	lsass
1886	28	12476	25768	0,25	1840	0	lsass
0	0	0	0	2,77	2808	0	Memory Compression
179	11	1448	11216	0,00	16568	1	netproctoolbox
648	17	18880	31808	0,38	1548	0	netproctoolbox
770	18	10712	19184	1,28	1216	1	netproctoolbox
414	26	4184	5264	0,61	1224	1	netproctoolbox
170	19	7492	19184	0,54	11144	1	netproctoolbox
177	17	8744	16480	0,00	4080	1	netproctoolbox
100	21	12608	17088	0,00	1080	1	netproctoolbox
148	18	2184	7440	0,00	15152	1	netproctoolbox
181	12	1008	24	0,00	1268	1	netproctoolbox

```
PS C:\Windows\system32> Get-process -Name opera
```

Após usarmos o comando git process. Usaremos o Git-process -Name opera, pois esse comando mostra todas as tarefas em execução que pertencem ao opera.



Handles	PM(K)	PM(K)	MS(K)	PM(K)	ID	SI	ProcessName
434	25	31464	51808	0,00	1936	1	opera
420	17	21456	120144	0,23	1952	1	opera
405	25	18912	55452	0,00	524	1	opera
371	16	18972	42692	0,03	3684	1	opera
558	43	31132	60632	1,14	4312	1	opera
1144	81	47496	21936	02,71	4300	1	opera
384	54	29488	41780	24,41	4776	1	opera
236	18	9564	21520	0,03	5236	1	opera
720	39	180316	226844	1,50	2672	1	opera
497	17	38920	153720	0,27	8888	1	opera
358	22	15284	28160	0,02	9160	1	opera
476	25	36788	49164	0,06	9992	1	opera
570	38	51232	39884	0,14	9272	1	opera
509	35	68484	184200	0,78	8864	1	opera
410	25	35112	34664	0,70	9916	1	opera
2792	99	283508	474668	23,95	10168	1	opera
552	31	53720	237284	1,86	10248	1	opera
417	27	18096	132556	0,23	10268	1	opera
385	19	12444	24380	0,05	10716	1	opera
527	28	65316	85264	0,62	10724	1	opera
421	26	21168	130364	0,27	10832	1	opera
597	44	265452	317924	12,25	10988	1	opera
557	44	201828	275440	12,60	11000	1	opera
483	16	54492	149396	0,19	11424	1	opera
510	16	172096	209228	7,06	11444	1	opera
529	35	71132	152728	0,26	11556	1	opera
648	33	63588	165140	0,27	11568	1	opera
417	25	21088	11868	0,00	11868	1	opera
601	45	166184	289460	0,81	12968	1	opera
350	22	14768	34580	0,02	15872	1	opera
414	27	70492	126024	0,15	16768	1	opera
495	35	67976	183308	11,06	16792	1	opera
610	49	181348	529172	44,22	17340	1	opera
509	41	79724	227280	1,51	17628	1	opera

Logo após de usar o comando podemos visualizar as tarefas executadas pelo navegador Opera

```
Administrative: Windows PowerShell

510 36 172896 289228 7,00 11556 1 opera
520 35 71152 81536 8,40 15756 1 opera
640 33 63584 105140 8,27 11568 1 opera
427 25 27008 51868 6,63 11864 1 opera
683 35 161984 309440 8,11 12664 1 opera
430 22 16168 44500 8,82 15872 1 opera
434 27 25492 118024 8,13 12568 1 opera
495 35 67076 183388 11,86 15792 1 opera
610 40 103148 193772 8,22 17360 1 opera
580 31 79224 222280 1,91 17628 1 opera

PS C:\Windows\System32> Stop-process -name "fotos"
O processo "foto" não é possível localizar um processo com o nome "fotos". Verifique o nome do processo e chame o cmdlet novamente.

PS C:\Windows\System32> Stop-process -name "fotos"
O processo "fotos" não é encontrado.

PS C:\Windows\System32> Stop-process -name "fotos"
O processo "fotos" não é encontrado.

PS C:\Windows\System32> Stop-process -name "photos"
O processo "photos" não é encontrado.

PS C:\Windows\System32>
```

Usamos este comando para encerrar um processo o exemplo usado foi (Stop-process -Name photos). A função dele foi encerrar o processo de execução do fotos.

A screenshot of a Windows command prompt window. The title bar reads "C:\Windows\System32". The command prompt shows the command `Stop-process -id 16672` entered at the prompt. The taskbar at the bottom shows the "Inicio" (Start) button, a search bar with the text "Buscar", and several application icons including File Explorer, Microsoft Edge, and the Task View button. The system tray on the right shows the date and time as "18/10/2025 13:12".

```

Administrator: Windows Eventlog
200 12 1616 7274 0,00 16660 0 svchost
111 0 1552 6280 0,00 13620 0 svchost
6885 0 44 5610 71,50 4 0 System
3445 /s 180216 2790 0,11 3956 1 SystemSettings
431 20 5256 24380 0,43 6882 1 SystemSettingsBroker
527 64 18764 21780 0,45 6508 1 TaskHost
385 22 6572 20000 0,45 7288 1 TaskHost
1121 66 57164 71932 0,31 17532 1 TextInputHost
1275 0 1506 2760 0,62 9760 1 vmacthlp
158 11 2276 10004 0,62 2446 1 UserNotificationCenter
152 9 1012 10616 0,80 5424 1 vmacthlp
500 27 43116 18020 0,83 1362 1 vmacthlp
1479 73 107372 93480 2,41 16676 1 WhatsApp
304 38 3100 16700 0,38 2612 1 WhatsApp
332 20 5116 21272 0,40 10719 1 WidgetsService
130 12 1644 8612 0,46 206 1 WinInit
286 14 2056 11700 0,11 1130 1 Winlogon
4404 120 880638 529736 183,66 13224 1 WinSxS
840 21 26164 14612 71,80 5220 1 WinVista

```

Usaremos uma variação do comando usado anteriormete, o comando `Stop-process -id 16672` , ele encerra um processo do programa fotos usando o número id do mesmo.

3.3. Android:

ADB SHELL PS :

```
PS Prompt de Comando
Microsoft Windows [versão 10.0.19045.6456]
(c) Microsoft Corporation. Todos os direitos reservados.

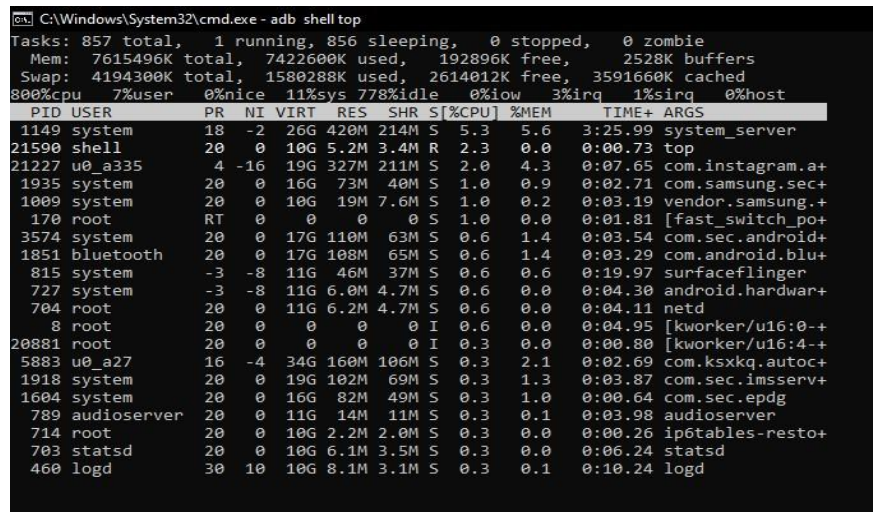
C:\Users\shokt>adb shell ps

USER      PID   PPID      VSZ    RSS  WCHAN      ADDR  S  NAME
root       1     0         0      0 0 0 0 0 0 0 S [init]
root       2     0    11856992 5224 0 0 0 0 0 0 S [kthreadd]
root       3     2         0      0 0 0 0 0 0 0 I [rcu_gp]
root       4     2         0      0 0 0 0 0 0 0 I [slub_flushq]
root       5     2         0      0 0 0 0 0 0 0 I [netns]
root       7     2         0      0 0 0 0 0 0 0 I [kworker/0:0H-fsverity_read_queue]
root       8     2         0      0 0 0 0 0 0 0 I [kworker/u16:0-blk_sec_common]
root       9     2         0      0 0 0 0 0 0 0 I [mm_percpu_wq]
root      11     2         0      0 0 0 0 0 0 0 S [rcu_tasks_kthread]
root      12     2         0      0 0 0 0 0 0 0 S [rcu_tasks_trace_kthread]
root      13     2         0      0 0 0 0 0 0 0 S [ksftirqd/0]
root      14     2         0      0 0 0 0 0 0 0 I [rcu_preempt]
root      15     2         0      0 0 0 0 0 0 0 S [rcub/0]
root      16     2         0      0 0 0 0 0 0 0 S [rcuc/0]
root      17     2         0      0 0 0 0 0 0 0 S [rcu_exp_gp_kthread_worker]
root      18     2         0      0 0 0 0 0 0 0 S [rcu_exp_pap_gp_kthread_worker]
root      19     2         0      0 0 0 0 0 0 0 S [migration/0]
root      20     2         0      0 0 0 0 0 0 0 S [idle_inject/0]
root      21     2         0      0 0 0 0 0 0 0 S [cpuhp/0]
root      22     2         0      0 0 0 0 0 0 0 S [cpuhp/1]
root      23     2         0      0 0 0 0 0 0 0 S [idle_inject/1]
root      24     2         0      0 0 0 0 0 0 0 S [migration/1]
root      25     2         0      0 0 0 0 0 0 0 S [rcuc/1]
root      26     2         0      0 0 0 0 0 0 0 S [ksftirqd/1]
root      28     2         0      0 0 0 0 0 0 0 I [kworker/1:0H-kblockd]
root      29     2         0      0 0 0 0 0 0 0 S [cpuhp/2]
root      30     2         0      0 0 0 0 0 0 0 S [idle_inject/2]
root      31     2         0      0 0 0 0 0 0 0 S [migration/2]
root      32     2         0      0 0 0 0 0 0 0 S [rcuc/2]
root      33     2         0      0 0 0 0 0 0 0 S [ksftirqd/2]
root      35     2         0      0 0 0 0 0 0 0 I [kworker/2:0H-kverityd]
root      36     2         0      0 0 0 0 0 0 0 S [cpuhp/3]
root      37     2         0      0 0 0 0 0 0 0 S [idle_inject/3]
root      38     2         0      0 0 0 0 0 0 0 S [migration/3]
root      39     2         0      0 0 0 0 0 0 0 S [rcuc/3]
root      40     2         0      0 0 0 0 0 0 0 S [ksftirqd/3]
root      42     2         0      0 0 0 0 0 0 0 I [kworker/3:0H-kblockd]
root      43     2         0      0 0 0 0 0 0 0 S [cpuhp/4]
root      44     2         0      0 0 0 0 0 0 0 S [idle_inject/4]
root      45     2         0      0 0 0 0 0 0 0 S [migration/4]
root      46     2         0      0 0 0 0 0 0 0 S [rcuc/4]
root      47     2         0      0 0 0 0 0 0 0 S [ksftirqd/4]
root      48     2         0      0 0 0 0 0 0 0 I [kworker/4:0-events]
root      50     2         0      0 0 0 0 0 0 0 S [cpuhp/5]
root      51     2         0      0 0 0 0 0 0 0 S [idle_inject/5]
root      52     2         0      0 0 0 0 0 0 0 S [migration/5]
root      53     2         0      0 0 0 0 0 0 0 S [rcuc/5]
root      54     2         0      0 0 0 0 0 0 0 S [ksftirqd/5]
root      57     2         0      0 0 0 0 0 0 0 S [cpuhp/6]
root      58     2         0      0 0 0 0 0 0 0 S [idle_inject/6]
root      59     2         0      0 0 0 0 0 0 0 S [migration/6]
root      60     2         0      0 0 0 0 0 0 0 S [rcuc/6]
root      61     2         0      0 0 0 0 0 0 0 S [ksftirqd/6]
root      64     2         0      0 0 0 0 0 0 0 S [cpuhp/7]
root      65     2         0      0 0 0 0 0 0 0 S [idle_inject/7]
root      66     2         0      0 0 0 0 0 0 0 S [migration/7]
root      67     2         0      0 0 0 0 0 0 0 S [rcuc/7]
root      68     2         0      0 0 0 0 0 0 0 S [ksftirqd/7]
```

comando adb shell ps é utilizado para listar todos os processos em execução no sistema Android. Ele exibe informações como o identificador do processo (PID), o processo pai (PPID), o usuário que o executa e o nome do processo ou aplicativo.

Esse comando é essencial para a inspeção e monitoramento do estado atual dos processos, permitindo identificar quais aplicações ou serviços estão ativos no momento e verificar possíveis sobrecargas no sistema.

ADB SHELL TOP :



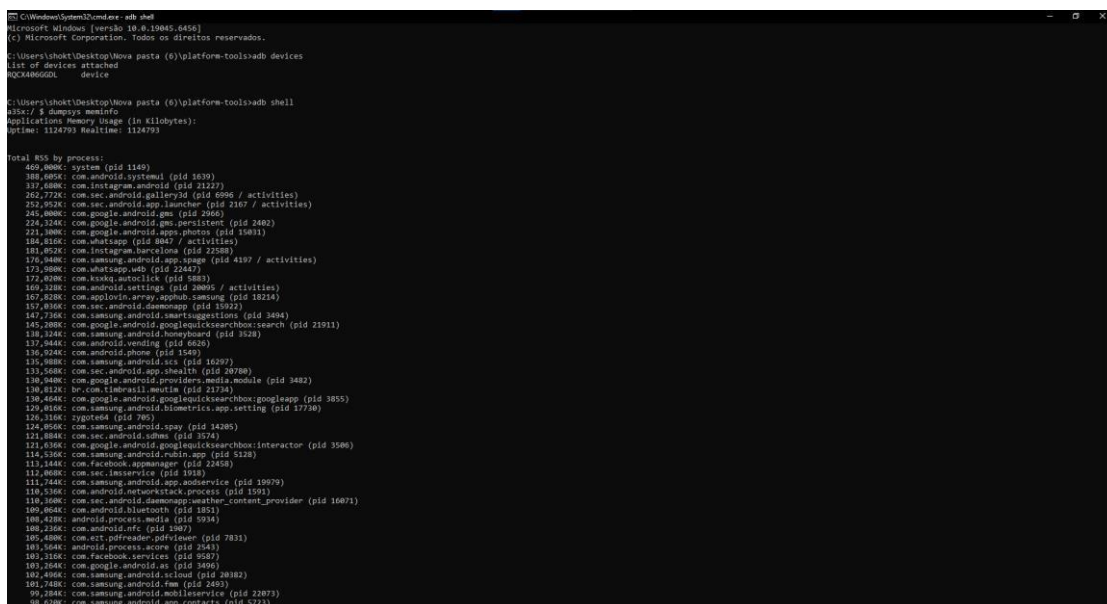
```
C:\Windows\System32\cmd.exe - adb shell top
Tasks: 857 total, 1 running, 856 sleeping, 0 stopped, 0 zombie
Mem: 7615496K total, 7422600K used, 192896K free, 2528K buffers
Swap: 4194300K total, 1580288K used, 2614012K free, 3591660K cached
800%cpu 7%user 0%nice 11%sys 77%idle 0%iow 3%irq 1%irq 0%host
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	ARGS
1149	system	18	-2	26G	420M	214M	S	5.3	5.6	3:25.99	system_server
21590	shell	20	0	10G	5.2M	3.4M	R	2.3	0.0	0:00.73	top
21227	u0_a335	4	-16	19G	327M	211M	S	2.0	4.3	0:07.65	com.instagram.a+
1935	system	20	0	16G	73M	40M	S	1.0	0.9	0:02.71	com.samsung.sec+
1009	system	20	0	10G	19M	7.6M	S	1.0	0.2	0:03.19	vendor.samsung.+
170	root	RT	0	0	0	0	S	1.0	0.0	0:01.81	[fast_switch_po+
3574	system	20	0	17G	110M	63M	S	0.6	1.4	0:03.54	com.sec.android+
1851	bluetooth	20	0	17G	108M	65M	S	0.6	1.4	0:03.29	com.android.blut
815	system	-3	-8	11G	46M	37M	S	0.6	0.6	0:19.97	surfaceflinger
727	system	-3	-8	11G	6.0M	4.7M	S	0.6	0.0	0:04.30	android.hardwar+
704	root	20	0	11G	6.2M	4.7M	S	0.6	0.0	0:04.11	netd
8	root	20	0	0	0	0	I	0.6	0.0	0:04.95	[kworker/u16:0-+
20881	root	20	0	0	0	0	I	0.3	0.0	0:00.80	[kworker/u16:4-+
5883	u0_a27	16	-4	34G	160M	106M	S	0.3	2.1	0:02.69	com.ksxkq.autoc+
1918	system	20	0	19G	102M	69M	S	0.3	1.3	0:03.87	com.sec.imsserv+
1604	system	20	0	16G	82M	49M	S	0.3	1.0	0:00.64	com.sec.epdg
789	audioserver	20	0	11G	14M	11M	S	0.3	0.1	0:03.98	audioserver
714	root	20	0	10G	2.2M	2.0M	S	0.3	0.0	0:00.26	ip6tables-resto+
703	statsd	20	0	10G	6.1M	3.5M	S	0.3	0.0	0:06.24	statsd
460	logd	30	10	10G	8.1M	3.1M	S	0.3	0.1	0:10.24	logd

O comando adb shell top exibe, em tempo real, o uso de CPU e memória de cada processo em execução. Assim como no Linux, ele permite acompanhar os processos que mais consomem recursos, apresentando estatísticas dinâmicas sobre o desempenho geral do sistema.

É amplamente utilizado para análises de desempenho e otimização de recursos, ajudando a identificar aplicativos que podem estar provocando lentidão ou consumo excessivo de energia.

DUMPSYS MEMINFO:



```
C:\Windows\System32\cmd.exe - adb shell
Microsoft Windows [vers o 10.0.18363.656]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\shakt\Desktop>adb devices
List of devices attached
8Q3A8060A    device

C:\Users\shakt\Desktop>adb shell
shakt/$ dumsys meminfo
Applications Memory Usage (in Kilobytes):
Optime: 1124793 Realtime: 1124793

Total RSS by process:
469,000K: system (pid 1149)
389,685K: com.android.systemui (pid 1639)
337,448K: com.instagram.android (pid 21227)
252,772K: com.sec.android.gallery3d (pid 6996 / activities)
252,092K: com.sec.android.app.launcher (pid 2167 / activities)
245,080K: com.google.android.gms (pid 2966)
224,324K: com.google.android.gms.persistent (pid 2460)
221,300K: com.google.android.apps.photos (pid 15011)
184,615K: com.whatsapp (pid 8847 / activities)
181,612K: com.instagram.brooklyn (pid 22508)
176,948K: com.samsung.android.app.spage (pid 4197 / activities)
172,000K: com.whatsapp (pid 22427)
172,038K: com.ksxkq.autoclick (pid 5883)
169,328K: com.android.settings (pid 28095 / activities)
167,825K: com.apollo.in.survey.splash.samsung (pid 19214)
157,016K: com.sec.android.daemonapp (pid 15922)
147,715K: com.samsung.android.smarttagengine (pid 3484)
145,280K: com.google.android.googlequicksearchbox:search (pid 21911)
138,234K: com.samsung.android.homepanel (pid 3528)
137,944K: com.android.wending (pid 6636)
136,924K: com.android.phone (pid 1549)
135,368K: com.samsung.android.scs (pid 16297)
133,568K: com.sec.android.app.shealth (pid 20700)
130,948K: com.google.android.providers.media.module (pid 3482)
130,612K: br.com.timbrasil.moutis (pid 21734)
130,404K: com.google.android.googlequicksearchbox:googleapp (pid 3855)
129,016K: com.samsung.android.biometrics.app.setting (pid 3730)
126,316K: rygotem4 (pid 785)
124,465K: com.samsung.android.spage (pid 14285)
121,884K: com.sec.android.sdmns (pid 3574)
121,655K: com.google.android.googlequicksearchbox:interactor (pid 3506)
114,536K: com.samsung.android.rubin.app (pid 5328)
113,144K: com.facebook.appmanager (pid 22458)
112,800K: com.sec.isservice (pid 3518)
111,744K: com.samsung.android.app.aodservice (pid 15979)
110,516K: com.samsung.android.networkstack.process (pid 1521)
110,100K: com.sec.android.daemonapp:weather_content_provider (pid 16071)
109,064K: com.android.bluetooth (pid 4853)
108,428K: android.process.media (pid 9394)
108,220K: com.android.rfc (pid 1307)
105,450K: com.st.nfrvader.pdrviewer (pid 7831)
101,504K: android.process.acore (pid 2543)
101,255K: com.facebook.service (pid 9587)
101,204K: com.google.android.as (pid 3496)
100,490K: com.samsung.android.rubin (pid 20182)
101,740K: com.samsung.android.rubin (pid 2493)
99,284K: com.samsung.android.mobileservice (pid 22873)
96,450K: com.samsung.android.app.contacts (pid 5723)
```

O comando adb shell dumsys meminfo fornece um relat rio detalhado sobre o uso de mem ria (RAM) de todos os processos do sistema ou de um aplicativo espec fico. Ele apresenta dados como a mem ria total utilizada, mem ria livre, buffers e cache.

Esse comando é fundamental para avaliar a eficiência da alocação de memória e identificar possíveis vazamentos (memory leaks), contribuindo para o diagnóstico de desempenho em dispositivos Android.

DUMPSYS CPUINFO

```
a35x:/ $ dumpsys cpuinfo
Load: 14.35 / 15.23 / 12.49
CPU usage from 122110ms to 73737ms ago (2025-10-17 23:44:09.894 to 2025-10-17 23:44:58.267):
12% 1149/system_server: 0.4% user + 0.4% kernel / faults: 1786 minor 14 major
1% 1630/com.android.systemui: 0.7% user + 0.3% kernel / faults: 368 minor
1% 1009/vendor.samsung.hardware.biometrics.fingerprint-service: 0.2% user + 0.7% kernel
0.9% 2258/com.instagram.barcelona: 0.6% user + 0.2% kernel / faults: 839 minor 1 major
0.8% 1935/com.samsung.sec.android.teegris.tui.service: 0.3% user + 0.5% kernel / faults: 6 minor
0.7% 170/fast_switch_post_worker_dm_cpucpl0: 0% user + 0.7% kernel
0.7% 2402/com.google.android.gms.persistent: 0.6% user + 0.1% kernel / faults: 724 minor 1 major
0.5% 20881/kworker/u16:4-blk_sec_common: 0% user + 0.5% kernel / faults: 2 minor
0.4% 704/netd: 0.1% user + 0.3% kernel / faults: 1192 minor
0.4% 18214/com.applivin.array.apphub.samsung: 0.3% user + 0.1% kernel / faults: 253 minor
0.3% 14/rcu_preempt: 0% user + 0.3% kernel
0.3% 19980/kworker/u16:3-blk_sec_common: 0% user + 0.3% kernel
0.3% 20934/kworker/u16:5-hlp_work: 0% user + 0.3% kernel / faults: 4 minor
0.3% 460/logd: 0.1% user + 0.1% kernel / faults: 38 minor
0.3% 1918/com.sec.imsservice: 0.1% user + 0.1% kernel / faults: 17 minor 1 major
0.3% 3574/com.sec.android.sdhms: 0.1% user + 0.1% kernel / faults: 113 minor
0.3% 815/surfaceflinger: 0.1% user + 0.1% kernel
0.2% 20889/adbd: 0% user + 0.2% kernel / faults: 622 minor
0.2% 192/simpleinteractive: 0% user + 0.2% kernel
0.2% 1591/com.android.networkstack.process: 0.1% user + 0% kernel / faults: 173 minor
0.2% 5883/com.kxxkg.autoclick: 0% user + 0.1% kernel / faults: 225 minor 3 major
0.2% 462/servicemanager: 0.1% user + 0% kernel
0.2% 755/vendor.samsung.hardware.health-service: 0% user + 0.1% kernel
0.2% 1549/com.android.phone: 0.1% user + 0% kernel / faults: 117 minor
0.2% 2494/com.samsung.android.smartsuggestions: 0.1% user + 0% kernel / faults: 1 minor 2 major
0.2% 22458/com.facebook.appmanager: 0.1% user + 0% kernel / faults: 295 minor
0.1% 1/init: 0.1% user + 0% kernel
0.1% 5592/com.samsung.android.wifi.ai: 0.1% user + 0% kernel / faults: 86 minor
0.1% 307/ueventd: 0.1% user + 0% kernel
0.1% 1851/com.android.bluetooth: 0% user + 0% kernel / faults: 18 minor 2 major
0.1% 14205/com.samsung.android.spay: 0% user + 0.1% kernel / faults: 21 minor
0.1% 703/statsd: 0% user + 0% kernel
0.1% 741/android.hardware.mediastack-service.exynos: 0% user + 0.1% kernel
0.1% 760/vendor.samsung.hardware.thermal@1.0-service: 0% user + 0.1% kernel
0.1% 727/android.hardware.graphics.composer@2.4-service: 0% user + 0% kernel
0.1% 789/audioserver: 0% user + 0% kernel
0.1% 954/kworker/u17:2-kbase_rm_poweroff_wait: 0% user + 0.1% kernel
0.1% 2167/com.sec.android.app.launcher: 0% user + 0% kernel / faults: 42 minor
0.1% 2543/android.process.acore: 0% user + 0% kernel / faults: 152 minor
0.1% 3520/com.samsung.android.honeyboard: 0% user + 0% kernel / faults: 25 minor
0.1% 4197/com.samsung.android.app.spagae: 0% user + 0% kernel / faults: 57 minor 3 major
0.1% 5934/android.process.media: 0% user + 0% kernel
0.1% 20095/com.android.settings: 0% user + 0% kernel / faults: 62 minor 206 major
0% 506/iod: 0% user + 0% kernel
0% 987/rild: 0% user + 0% kernel
0% 5723/com.samsung.android.app.contacts: 0% user + 0% kernel / faults: 9 minor
0% 6658/com.google.android.webview:sandboxed_process0:org.chromium.content.app.SandboxedProcessService0: 0% user + 0% kernel
0% 22512/com.facebook.system: 0% user + 0% kernel / faults: 132 minor
0% 22534/com.samsung.android.sm.provider: 0% user + 0% kernel / faults: 124 minor 28 major
0% 171/fast_switch_post_worker_dm_cpucpl1: 0% user + 0% kernel
0% 181/thermal_BIG: 0% user + 0% kernel
0% 480/psimon: 0% user + 0% kernel
0% 523/android.system.suspend-service: 0% user + 0% kernel
0% 813/perfmond: 0% user + 0% kernel
0% 17730/com.samsung.android.biometrics.app.setting: 0% user + 0% kernel / faults: 17 minor
0% 20959/com.samsung.android.iol: 0% user + 0% kernel / faults: 60 minor 220 major
0% 22073/com.samsung.android.mobileservice: 0% user + 0% kernel / faults: 19 minor 17 major
0% 13/ksoffirqd/0: 0% user + 0% kernel
0% 26/ksoffirqd/1: 0% user + 0% kernel
```

O comando adb shell dumpsys cpuinfo exibe o uso de CPU por processo ou aplicativo, em forma de porcentagem. Ele é utilizado para analisar o consumo de processamento de cada app, auxiliando na identificação de gargalos de desempenho e no controle da carga total do processador.

Em contextos de gerenciamento de processos, esse comando permite monitorar a eficiência da execução das tarefas e garantir uma distribuição equilibrada dos recursos do sistema.

ps -T -p <PID> :

```
a35x:/ $ ps -T -p 1
USER      PID     TID  PPID      VSZ    RSS  WCHAN      ADDR S  CMD
root        1         1      0    11056992 5880  0           0 S  init
root        1      384      0    11056992 5880  0           0 S  init
root        1      385      0    11056992 5880  0           0 S  init
a35x:/ $ _
```

O comando ps -T -p <PID> lista todas as threads associadas a um processo específico, identificando cada thread com seu próprio identificador (TID) e estado atual.

Esse comando é importante para compreender a estrutura interna de execução de um processo, especialmente em aplicações que utilizam múltiplas threads para realizar tarefas

simultâneas. Ele permite uma visão mais detalhada do funcionamento interno de um processo no Android.

PM LIST PACKAGES:

```

C:\Users\shokt>adb shell
Microsoft Windows [versão 10.0.19045.6456]
(c) Microsoft Corporation. Todos os direitos reservados.
C:\Users\shokt>adb devices
List of devices attached
MQ2XAM602N    device
C:\Users\shokt>adb shell
shokt / $ pm list packages
package:com.sec.android.allServiceNodeApp
package:br.com.sml.beneficiario
package:com.samsung.oda.service
package:com.google.android.overlay.modules.permissioncontroller.framework
package:com.sec.android.lgt
package:com.samsung.android.providers.trash
package:com.samsung.android.app.telephony.esimclient
package:com.samsung.android.vcamerasettings
package:com.samsung.android.mdservice
package:com.samsung.android.email.provider
package:com.samsung.android.dms
package:com.samsung.android.dreams.phototable
package:com.samsung.android.mcfds
package:com.samsung.android.smartface.overlay
package:com.samsung.android.knox.kpccore
package:com.samsung.internal.systemui.navbar.sec_gestural_no_hint
package:com.android.companiondevicemanager
package:com.android.cts.priv.ctsshin
package:com.samsung.android.app.spage
package:com.samsung.android.lsol
package:com.android.providers.downloads
package:br.com.cruzeirodovirtual
package:com.sec.android.app.parser
package:com.android.bluetoothdiservice
package:com.facebook.appmanager
package:com.android.credentialsmanager
package:com.samsung.shop
package:com.samsung.android.app.settingsmanager
package:com.google.android.printservice.recommendation
package:com.sec.facatfunction
package:com.google.android.castiveportallogin
package:com.sec.android.app.vpreload
package:com.google.android.networkstack
package:com.samsung.android.wifi.suupgradeoffload.resources
package:com.inovatecologia.cartelrinha
package:com.android.keychain
package:com.google.android.overlay.gmsconfig.asi
package:com.microsoft.office.outlook
package:com.samsung.liservice
package:com.samsung.android.app.surphonetypec
package:com.samsung.android.galaxy
package:com.android.shell
package:com.kilno.subwaysurf
package:com.samsung.android.accessibility.talkback
package:com.google.android.overlay.personalization.services
package:com.sec.android.app.magnifier
package:com.google.android.overlay.modules.healthfitness.framework
package:com.android.bookmarkprovider
package:com.samsung.android.wifi.softapathree.resources
package:com.samsung.android.app.contacts
package:com.samsung.android.samsungpass
package:com.snp.snplogin
package:com.android.sharedstoragebackup

```

O comando `adb shell pm list packages` é utilizado para listar todos os pacotes de aplicativos instalados no dispositivo Android, incluindo tanto aplicativos do usuário quanto os nativos do sistema.

Esse comando pertence ao Package Manager (pm) e é essencial para o gerenciamento de aplicativos e permissões, sendo possível identificar nomes de pacotes necessários para execução de outros comandos, como `force-stop`, `clear` ou `install`.

O sistema Android disponibiliza diversos comandos para o gerenciamento de processos. Entre os principais, destacam-se: **ps**, responsável pela listagem e inspeção de processos em execução; **kill**, utilizado para encerramento de processos; **dumpsys**, que permite monitorar o uso de recursos do sistema, como CPU e memória; e **pm**, voltado ao gerenciamento de pacotes, permissões e aplicativos instalados.

4. ANALISE CRITICA:

4.1. Tabela comparativa:

Filosofia de controle		
Linux	Windows	Android
Se baseia no conceito de prioridade “nice”, representada por valores entre -20 e +19. Quanto menor o valor, maior a prioridade do processo na CPU. Também tem o CFS (Completely Fair Scheduler), que distribui o tempo de processamento de forma justa, ajustando dinamicamente as prioridades conforme a carga do sistema.	Utiliza níveis de prioridade numéricos (que vão de 0 a 31) e classes de prioridade [tais como: Idle; Normal; High e Realtime]. O Windows Scheduler determina a execução com base nessas classes, priorizando processos mais críticos e os ajustando conforme a resposta do sistema e a interação do usuário	Segue o ciclo de vida dos aplicativos, dividindo esses entre os estados de foreground, background e stopped. O sistema define automaticamente a prioridade de cada processo com base no uso atual, encerrando ou suspendendo tarefas para otimizar desempenho e consumo de energia.
Granulidade de monitoramento		
Linux	Windows	Android
Oferece um nível muito detalhado de monitoramento, permitindo visualizar processos, consumo de CPU, memória, I/O, identificadores (PID e PPID) e diversos outros parâmetros em tempo real. Ferramentas como top, htop e o diretório /proc fornecem acesso direto às informações do kernel, garantindo um controle preciso e técnico sobre o sistema	Possui uma granularidade intermediária/média, exibindo informações completas sobre uso de CPU, memória, identificadores e serviços associados. O Gerenciador de Tarefas e o Process Explorer permitem uma visão detalhada dos processos, embora com menor acesso direto ao núcleo do sistema em comparação ao Linux.	Apresenta um nível de monitoramento mais restrito, voltado principalmente para aplicativos e serviços. Ferramentas como o Android Studio Profiler e o ADB shell permitem acompanhar uso de CPU, memória além de rede e threads de cada app, com foco em desempenho e consumo de recursos, e não no monitoramento global do sistema.
Sintaxe de comandos		
Linux	Windows	Android
1- ps aux => lista todos os processos; 2- top / htop => monitoramento dinâmico; 3- nice / renice => altera prioridade.	1- tasklist => lista processos; 2- taskkill /PID <id> => encerra processo; 3- wmic process list => exibe informações detalhadas.	1- adb shell ps => lista os processos ativos no dispositivo; 2- adb shell top => exibe o uso da CPU em tempo real; 3- adb shell kill <PID> => encerra um processo específico.

4.2. Discussão:

A forma como cada sistema gerencia seus processos mostra bem o objetivo principal de cada um.

No Linux, a ideia é dar o máximo de controle para o usuário. Ele permite ajustar manualmente a prioridade dos processos com o valor “nice” e usar comandos como `renice` para mudar o comportamento do sistema caso haja alguma necessidade. Isso é ótimo para servidores ou sistemas que precisam de alto desempenho, já que usuário pode escolher o que terá mais ou menos atenção d da S.O.

O Windows segue um caminho mais equilibrado. Ele também deixa o usuário mudar prioridades (pelo Gerenciador de Tarefas ou por comandos como `wmic`), mas o sistema faz vários ajustes automáticos conforme o que está sendo usado no momento. Assim, ele tenta manter um bom desempenho e facilidade de uso, sem precisar que o usuário entenda muito sobre processos.

Já o Android tem um foco bem diferente: economizar energia e usar os recursos da melhor forma possível. Por conta disso, encerra processos em segundo plano quando eles não são mais necessários, mantendo prioridade apenas para os aplicativos que estão em uso ativo [foreground]. Nesse caso, o usuário não controla diretamente as prioridades, já que o sistema faz tudo de forma automática para garantir eficiência e duração da bateria.

No fim, dá pra dizer que Linux e Windows oferecem mais liberdade de controle, enquanto o Android aposta na automação, simplicidade e no uso inteligente da energia, dando grande importância entre o equilíbrio e desempenho do sistema.

5. CONCLUSÃO

Com base nos processos realizados acima, foi possível compreender as diferenças no gerenciamento de processos e threads entre os sistemas Windows, Linux e Android. Onde os três sistemas operam de uma forma diferente no controle de tarefas, no qual cada um prioriza pontos diferentes como desempenho, segurança e praticidade.

Na questão do Linux, o sistema se mostra mais completo e voltado para quem precisa de controles mais detalhados e de estabilidade em ambientes maiores. Já o Windows é mais simples de usar e oferece um gerenciamento automático que facilita o dia a dia do usuário. No Android, que modifica o estilo do Linux para funcionar em aparelhos moveis, garante que cada aplicativo funcione de forma isolada, o que aumenta a segurança e evita acessos indesejados a outras partes do sistema.

De maneira final, é possível enxergar a maneira com que cada sistema lida com o gerenciamento de processos e suas execuções, mostrando de forma prática qual e a sua importância no mantimento de uma boa funcionalidade e de equilíbrio nas tarefas em diferentes sistemas

6. AUTOAVALIAÇÃO:

Durante a realização dessa fase do relatório, cada membro teve a sua parte dividida, onde todos cooperaram entre si e fazendo o trabalho de forma organizada. Dessa vez enquanto fizemos o trabalho, não enfrentamos problemas na execução dos objetivos do relatório, e adquirimos conhecimentos sobre o que é o gerenciamento de processos, como ele funciona e quais são as suas funções.